

The Judgment Ledger — Solution Case Study (Draft)

1. How I identified the problem and reimaged the system

Most deploy-approval systems and most incident postmortems exist as two separate processes, built by different teams, running on different tools, rarely talking to each other. A deploy gate asks "should this ship right now." A postmortem asks "what actually happened after it shipped." The two are obviously related — the whole reason a gate exists is to prevent the kind of thing a postmortem investigates — but almost no organization treats them as one continuous system. The approval decision and the evidence of whether that decision was right live in different places, checked by different people, on different schedules. That gap is where a real failure mode lives: approval policy calcifies. A category gets marked "requires human review" once, early, when evidence is thin, and it often stays that way indefinitely — not because anyone re-evaluated it and found it still risky, but because nobody built a mechanism to revisit it. Meanwhile, the postmortem process, however good it is, has no formal channel back into what the approval gate actually checks next time. The lessons stay in a document. The gate stays static.

The reimagination was to stop treating approval and postmortem as two features and instead build them as one continuously-written evidence record — the Judgment Ledger — where every deploy decision and every real outcome write to the same object, and where that same object is what future decisions are checked against. Not a smarter approval bot. A structural answer to why approval policy and lived evidence normally never talk to each other.

2. Key agents, responsibilities, and human-AI interactions

The system is three agents, each with a single, non-overlapping responsibility, and two distinct places where a human holds real authority, not ceremonial sign-off.

Intake Agent — reads an incoming change, classifies its risk category, computes its blast radius against a service dependency map, and pulls matching history from the Ledger. It has zero decision-making authority. It hands a structured summary downstream and explicitly flags anything it can't confidently resolve — an ambiguous category, a leaked credential — rather than guessing. This agent never touches the word "approve."

Threshold Agent — the only agent allowed to make a routing decision. It checks whether the change's category currently sits inside a human-approved trust boundary, and how strong the evidence behind that boundary actually is. If the boundary and evidence support it, it auto-approves and ships. If not, it escalates to a human Senior Approver, and it never expands its own trust boundary under any circumstance — that authority belongs to a human, always, structurally, not as a design suggestion.

Reconciliation Agent — runs after deploy, watches real outcomes, and writes the honest verdict back to the Ledger. If a bad outcome lands in a category that was auto-approved, this agent immediately and automatically tightens that category back to requiring human review. This is a one-directional circuit breaker:

it can only make the system more cautious on its own. Restoring trust is never something an agent can do alone.

Where humans hold real authority: a Senior Approver reviews individual escalated deploys, with the agent's full reasoning and evidence attached, not a bare yes/no request. Separately, on a weekly cadence, Engineering Leadership reviews aggregated evidence and approves or rejects proposed changes to which categories are trusted — the only place trust is ever allowed to expand. These are two genuinely different checkpoints, at two different frequencies, because they answer two different questions: "should this one deploy ship" versus "should this whole category be trusted going forward."

3. Build: implementation, testing, and integration using Yoxa

The system was built in Yoxa as three agent columns feeding off one trigger, with tool-level sub-steps handling the concrete integrations: version control access, a service dependency graph, the Ledger itself, and the live trust boundary configuration. Each agent's persona (role, behavioral rules, tone) was kept separate from its step instructions (the concrete task at hand), following the platform's own guidance that conflating the two wastes prompt space and muddies both.

Because the fast loop and the slow loop run on genuinely different schedules — every deploy versus once a week — and the platform doesn't support native branching between two independent triggers, the two loops were implemented as one trigger carrying an explicit mode flag (`[trigger_type: deploy]` or `[trigger_type: policy_review]`), with every step instructed to check that flag and no-op if it isn't its turn. This was a deliberate workaround for a real platform constraint, not an accident — and it's documented here because it's a design decision in its own right, not just an implementation detail.

Reference data — mock Ledger history, category classification rules, the current trust boundary configuration, and a service dependency map — were built as structured documents and attached per-agent, per-tool, so each tool reads only the specific data it actually needs rather than everything being visible to every step. Output formats for every agent handoff were explicitly schema'd, then rewritten a second time into plain, human-readable sentence templates once early testing showed the raw field-based outputs were unreadable to anyone outside the build.

4. Evaluation: reliability, robustness, consistency, and agent behaviour

The system was tested against eight scenarios deliberately chosen to go past the happy path: an auto-approve baseline, an escalation with thin evidence, an ambiguous classification that spans two categories, a leaked credential in a diff, the circuit breaker actually firing after a confirmed problem, an inconclusive slow-feedback outcome, and both directions of the weekly policy review (holding steady, and recommending expansion).

Results, honestly: the auto-approve path shipped correctly and reconciled as confirmed safe, with fast and slow feedback both within baseline. The ambiguous-classification test correctly refused to force a category, reported 0% confidence rather than a fabricated number, and escalated with the ambiguity itself — not thin evidence — stated as the reason. The weekly policy review correctly held every under-evidenced category, correctly distinguished "insufficient sample size" from "small sample but a severe correction," and correctly

refused to make any configuration change on its own, treating its own output as a proposal awaiting explicit human approval, restating that boundary three separate times in its own reasoning.

Testing also surfaced two real bugs, which are included here as evidence of iteration rather than smoothed over:

Early builds had the deploy-trigger step sequenced *before* the human-approval step, meaning a Senior Approver's decision had no effect on outcome by the time it was made. Caught by deliberately testing both an approve and a block response to the same escalation and finding identical results — a genuine before/after fix, not a cosmetic one.

A second bug initially looked like the same issue reappearing — approvals still weren't affecting outcomes — but turned out to be a different root cause entirely: the deploy-trigger step was a Simulated Output tool, meaning it was returning fixed mock data regardless of any upstream input, by design, since no real deploy API is connected yet. Distinguishing "this is broken" from "this is correctly behaving like a placeholder for a not-yet-built integration" was itself part of the evaluation process.

The honest limitation, stated plainly rather than glossed over: because the deploy-trigger step remains a Simulated Output pending real API integration, the full approve-and-actually-ship path cannot be end-to-end validated inside Yoxa alone yet. Everything upstream of it — classification, evidence-weighting, escalation, the human decision itself — is confirmed working correctly.

5. Key iterations, learnings, and design decisions — the story of why, not just what

The single biggest pivot in this project came from a faculty critique that the concept was "meaningless" — that adaptive trust systems, error budgets, and circuit breakers already exist and are well documented, and that this was simply reinforcement learning re-described. That critique was correct on its face, and the honest response was to agree with it directly rather than defend the mechanism. The *mechanism* — a policy that tightens automatically on bad outcomes and loosens only with deliberate evidence — is not new. SRE error-budget practice and adjustable-autonomy research have covered this ground for years.

What followed was a genuine narrowing of the claim, grounded in actual research rather than a rhetorical dodge. Current 2026 research on human-AI oversight boundaries explicitly names this space as unresolved — one systematic review found full AI task delegation remains realistic for only 0-20% of tasks, with current frameworks still unable to characterize that boundary precisely. Industry guidance separately names "are escalation mechanisms actually being used, or do they exist only on paper" — rubber-stamping — as a specific, recurring failure mode in real deployments. And the EU AI Act's August 2026 enforcement deadline makes demonstrable human oversight a legal requirement, which is part of why this space is being actively worked on right now rather than five years ago.

The revised, defensible claim: this system doesn't invent adaptive trust. It takes a specific, structural position on the part of that space that's still unresolved — how to keep an escalation step from decaying into a formality as volume grows — by making write-access to the shared trust configuration asymmetric by design, not just by policy. A category can be tightened automatically and immediately by evidence alone. It can only ever be loosened by a human, deliberately, on a schedule, with the evidence trail attached. That asymmetry is the actual contribution, and it's visible not just in the backend logic but in the interface itself: the escalation

review screen gates its own approve button behind having opened the one piece of evidence that matters most, and the weekly review screen shows a category with no action buttons at all when there's nothing worth a human's attention — the interface enforces the same discipline the backend does.

A second, smaller but real decision: the system deliberately does not give Engineering Leadership override authority to bypass the Threshold Agent's per-deploy decisions directly. That option was considered and rejected, because it would recreate the exact rubber-stamping failure the system exists to prevent, just one level higher up the chain.

6. Is this worth building?

Clear value for the user. The direct beneficiaries are engineers waiting on a deploy and the Senior Approvers reviewing escalations — both currently operate inside systems where "requires approval" is a permanent, unchanging label with no mechanism to revisit it as real evidence accumulates. This system gives both groups a boundary that can move in response to actual outcomes, in both directions, without either side having to manually renegotiate policy.

Credible real-world applicability. Every organization running continuous deployment already has some version of an approval gate and some version of a postmortem process; almost none have unified them into one evidence object. This isn't a hypothetical use case — it's a structural gap in a process nearly every engineering org already runs, which means the applicability isn't speculative, it's a retrofit onto something that already exists everywhere.

Originality and differentiation. Honestly stated: the underlying adaptive-trust mechanism is not original, and claiming otherwise wouldn't survive scrutiny. What's original is the specific asymmetric-authority implementation, aimed squarely at a documented, currently-unresolved gap — keeping human oversight meaningful rather than theatrical as an agentic system scales — and expressed consistently across the backend logic, the agent design, and the interface itself, rather than solved in only one layer and ignored in the others.